CSE6234 Software Design

Software Design Proposal and Software Design Pattern Document

HealthTech

Development of a Machine Learning-Based System for Cardiopulmonary Sound Separation

Group 1

Tutorial Section: TT5L

Name	Student ID
AL-SALOUL, ASHRAF ALI HUSSEIN	1221303805
AHMAD AKMAL ASYRAAF BIN SHAMS	242UC244CJ
RESHMA A/P KRISHNAMURTHY	243UC247KW
SHARWIN A/L R SIVALINGAM	241UC241C1

May 8, 2026

Contents

Abstract / Executive Summary

This report presents a software design proposal and design pattern document for a HealthTech cardiopulmonary sound separation prototype. The proposed system accepts mixed WAV audio, uses NeoSSNet to separate the recording into heart sound and lung sound outputs, stores structured metadata and processing records in SQLite, and keeps uploaded and generated audio files in local storage.

The report focuses on software design rather than clinical validation. It documents the problem statement, objectives, system scope, UML models, requirements, design principles, and selected design patterns. FastAPI is used as the backend API layer, SQLite supports jobs, results, logs, and metadata, and the local filesystem stores audio artifacts and model files. The Facade, Strategy, and Factory Method patterns are proposed to simplify the separation workflow, support future algorithm extension, and centralise model or algorithm creation.

Introduction

1.1 Executive Summary / Abstract

This report presents a software design proposal and design pattern document for a cardiopulmonary sound separation system. The proposed system is a HealthTech prototype that accepts mixed WAV audio and separates it into heart sound and lung sound outputs using NeoSSNet. The project is designed as a local web-based application with a FastAPI backend, a simple HTML/CSS/JavaScript interface, SQLite database storage for metadata and processing records, and local filesystem storage for uploaded and generated audio files.

The purpose of the system is to demonstrate how machine learning inference can be integrated into a reusable software system rather than being kept only as an isolated model script. The application supports upload, validation, real separation, output preview, download, and processing history. This document focuses on software design quality, including requirements, UML modelling, database structure, layered architecture, and design patterns. The prototype is not a clinical diagnosis system and does not claim clinical validation. Instead, it is intended to show a practical and explainable software design for cardiopulmonary sound separation.

1.2 Background / Introduction

Cardiopulmonary sound recordings often contain both heart and lung components because the two sound sources are captured from nearby areas of the chest. In a mixed recording, the sound of one organ can interfere with analysis of the other. This creates a useful software engineering problem: the system must accept an input audio file, apply a sound separation model, produce separate heart and lung audio outputs, and keep the processing records traceable for review.

Recent research has shown that computational methods can support heart and lung sound processing, including source separation and deep learning-based heart sound analysis. However, a model alone is not enough for a usable application. A student, lecturer, researcher, or demonstration user needs a complete workflow that includes file upload, validation, inference, storage, result preview, download, error handling, and history. For this reason, the project is framed as a software design assignment that connects machine learning with maintainable backend and frontend architecture.

The proposed system uses NeoSSNet as the core separation model and stores model-related files in local storage. SQLite is used to store structured data such as uploaded audio metadata, model records, separation jobs, result paths, metrics, and logs. The actual WAV audio files are stored on the filesystem so that the database remains lightweight and easy to inspect. This separation between database records and audio artifacts supports a practical local deployment suitable for a Final Year Project prototype.

1.3 Problem Statement

Mixed cardiopulmonary audio is difficult to use directly when the target task requires separate heart and lung sound outputs. If a system only stores uploaded mixed audio, users cannot easily preview the isolated components or use them for later analysis and demonstration. At the same time, if separation is implemented only as a standalone script, the workflow may lack upload handling, database tracking, result retrieval, logging, and a clear user interface.

The project therefore needs a reusable software system that can connect audio upload, NeoSSNet inference, file storage, SQLite metadata, and browser-based result viewing. The design must remain simple enough for a student project, but structured enough to explain through software design concepts, UML diagrams, and design patterns. The system must not present separated audio as clinical diagnosis; it should be described as a prototype that demonstrates automated cardiopulmonary sound separation and supporting software architecture.

1.4 Project Objectives

The objectives of this project are:

1. To design a local web-based system that accepts mixed cardiopulmonary WAV audio through a browser interface.
2. To validate uploaded audio files before they are stored or processed by the backend.
3. To integrate NeoSSNet as the real cardiopulmonary sound separation model for generating heart and lung output audio.
4. To save separated heart and lung WAV files in local output folders for preview and download.

5. To store structured metadata, job records, result paths, logs, and optional evaluation metrics in SQLite.
6. To provide result viewing, audio preview, download support, and processing history through the web interface.
7. To document a maintainable software design using UML diagrams, requirements, design principles, and selected design patterns.

1.4.1 Literature Review

Cardiopulmonary sound analysis studies the information contained in heart and lung recordings, where useful physiological sounds may overlap because both sources are captured from the chest area. In practical recordings, the heart component can interfere with lung sound analysis, while lung sounds and breathing noise can also obscure heart sound patterns. Sound separation is therefore an important preprocessing and application problem because it attempts to recover clearer heart and lung signals from a mixed recording. For the proposed HealthTech prototype, the literature is relevant not only because it motivates the use of machine learning for heart and lung sound separation, but also because it shows the need for a usable software system around the algorithm. A practical system must support upload, validation, inference, storage, preview, download, and processing history rather than treating separation as an isolated research script.

Deep learning-based separation is the closest research theme to the proposed system because the prototype uses NeoSSNet as its core model. Poh et al. (2024) proposed NeoSSNet for real-time neonatal chest sound separation, with the aim of separating mixed chest sound recordings into heart and lung components. The method follows an audio source separation approach in which a waveform is encoded into a learned representation, separation masks are estimated, and separated outputs are reconstructed. This is directly aligned with the system goal because the application must produce two usable audio files: one heart output and one lung output. NeoSSNet also supports the project from a software design perspective because its inference process can be placed inside a dedicated machine learning layer and called by the backend service layer. However, the research contribution mainly concerns the model and separation performance. It does not by itself provide a complete application workflow for user upload, database-backed job tracking, browser preview, or maintainable backend organization.

A second theme is hybrid separation using signal processing and decomposition methods. Sun et al. (2024) studied a DAE–NMF–VMD method for separating heart and lung sounds. This approach combines deep autoencoder-based representation learning, nonnegative matrix factorization, and variational mode decomposition. Compared with a pure neural separation model such as NeoSSNet, the DAE–NMF–VMD method shows that cardiopulmonary sound separation can also be designed as a multi-stage processing pipeline that combines learned features with mathematical decomposition and denoising. This is useful for the proposed project because it shows that future versions of the system may need to compare different separation approaches. Different algorithms may require different preprocessing steps, configuration parameters, model files, and output handling. Therefore, the software should avoid embedding one algorithm too deeply into the route or controller logic. A modular design with an interchangeable algorithm interface is better suited for future experimentation while keeping the current prototype focused on NeoSSNet.

A broader theme is the role of deep learning in heart sound analysis. Zhao et al. (2024) reviewed deep learning techniques, datasets, and applications for heart sound analysis, including preprocessing, feature extraction, model development, and potential clinical application areas. This review is important because it places the project within a wider research area where machine learning is increasingly used to process phonocardiogram and related biomedical sound data. At the same time, the review also highlights issues such as dataset limitations, generalisation, and the need for further refinement before broad clinical use. These points are important for this report because the proposed system should not be presented as a clinically validated diagnostic tool. It is more accurately described as a proof-of-concept HealthTech software prototype that demonstrates cardiopulmonary sound separation and the supporting software architecture needed to make the workflow usable and traceable.

Taken together, these studies show that cardiopulmonary sound separation and heart sound analysis are active research areas with several promising algorithmic directions. NeoSSNet supports the feasibility of deep learning-based heart and lung sound separation, DAE–NMF–VMD shows the value of hybrid and decompositional approaches, and broader heart sound analysis research explains why careful preprocessing, dataset handling, and model evaluation matter. The main gap for this software design report is that existing work focuses strongly on algorithms, while less attention is given to reusable application architecture for a local prototype. In particular, the literature does not fully address database-backed processing history, structured metadata, user upload workflow, separated audio preview and download, local file storage, logging, and maintainability through design patterns. The proposed system responds to this gap by combining FastAPI for the backend API, SQLite for metadata and job records, local filesystem storage for uploaded and separated WAV files, and NeoSSNet for real separation. This design connects research-level separation models with a practical software workflow that can be demonstrated, inspected, and extended in a student software design assignment.

1.5 Project and System Scope

1.5.1 In Scope

The project scope covers a local prototype that accepts mixed WAV audio, validates the file, stores the uploaded audio in a local folder, runs NeoSSNet-based separation, generates separated heart and lung WAV outputs, stores output files locally, records metadata and job status in SQLite, and provides browser-based preview, download, and processing history. The report also covers software design artefacts such as use case modelling, class modelling, database modelling, requirements, design principles, and design patterns.

The dataset and runtime storage are treated as separate concerns. Dataset files such as HLS-CMDS data are used for development or evaluation purposes, while runtime uploads and generated outputs are stored under the application storage folders. This separation avoids mixing training or testing data with user-uploaded demo files.

1.5.2 Out of Scope

The project does not aim to provide medical diagnosis, clinical decision support, hospital deployment, or validated clinical accuracy. It does not claim that separated audio is suitable for real clinical use without further testing and approval. Authentication, cloud deployment, multi-user role management, and production-scale monitoring are also outside the current scope unless they are added in a later phase.

Training a new model from runtime uploads is also outside the current system scope. Runtime uploads are used for inference and demonstration only. The current design focuses on integrating real NeoSSNet inference into a clean software system that is practical for local execution and clear enough to defend in a software design presentation.

System Overview

The proposed system is a local web-based prototype for cardiopulmonary sound separation. A user uploads a mixed WAV audio file through the browser interface, and the FastAPI backend validates the upload, stores the original file, creates database records, runs NeoSSNet separation, saves the separated heart and lung audio files, and returns results for preview and download. SQLite is used for structured records such as uploaded audio metadata, model information, separation jobs, result paths, metrics, and logs, while local folders are used for the actual audio files and model checkpoints.

The following diagrams describe the system from different software design viewpoints. The use case diagram presents the system functions from the user's perspective, the class diagram shows the planned software components, the entity relationship diagram shows the database structure, and the object diagram gives an example of one completed separation job.

2.1 Generic Use Case

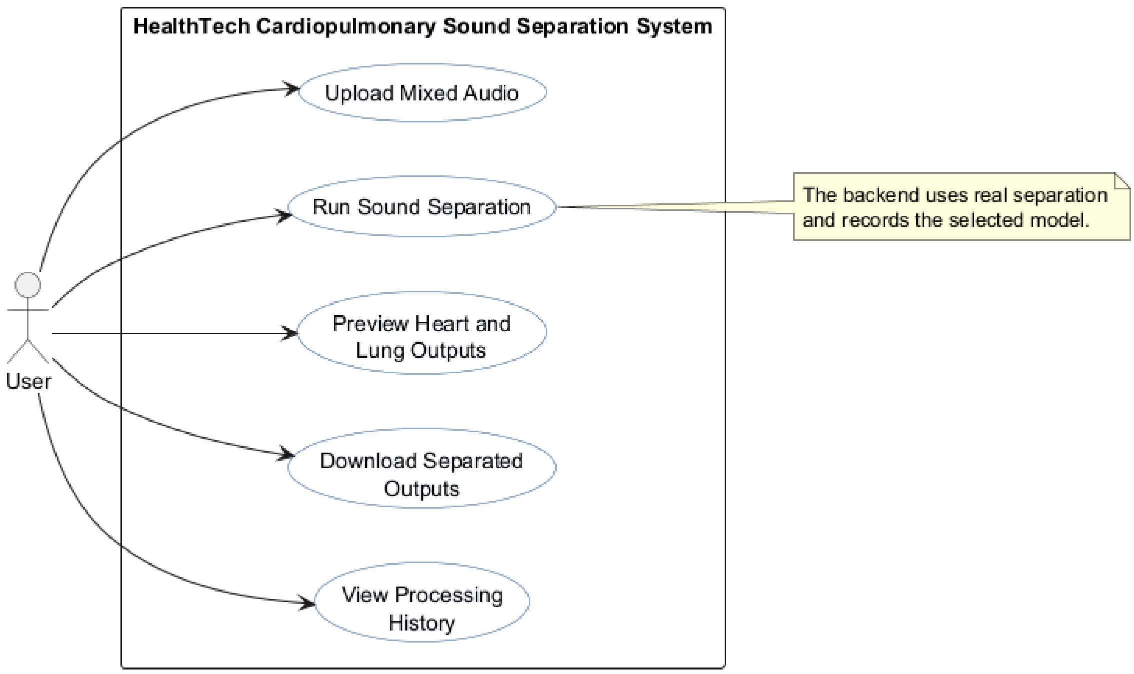

Figure 2.1: Generic Use Case Diagram

Figure 2.1 shows the main actions available to a single generic actor, the User. The diagram is intentionally simple because the same user-facing workflow can be demonstrated by a student, lecturer, examiner, or researcher without changing the system boundary. The central use cases are uploading mixed audio, running cardiopulmonary sound separation, previewing separated heart and lung outputs, downloading separated outputs, and viewing processing history.

The simplified diagram keeps internal responsibilities such as validation, model selection, metadata storage, and logging out of the actor view. Those responsibilities are still required by the backend, but they support the visible workflow rather than appearing as separate user goals. This makes the use case model easier to read while still showing how upload, separation, preview, download, and history are connected.

2.1.1 Use Case Specifications

A use case describes how an actor interacts with a system to achieve a specific goal. In software design, use cases are useful because they clarify system boundaries, user intentions, required inputs, normal workflows, and failure handling before detailed implementation begins. In the following use case specifications, the main course represents the normal successful flow, the alternate course represents another valid path through the same use case, and the exception courses describe error or failure conditions that the system should handle.

Table 2.1: RML-Style Use Case Specification for UC_001

Field	**Specification**
Name	Upload Mixed Audio
ID	`UC_001`
Description	The User uploads a mixed cardiopulmonary WAV file through the web-based interface so that it can be validated, stored, and prepared for separation.
Actor	User
Organization Benefits	Supports the first step of the prototype workflow by allowing mixed audio recordings to enter the system in a controlled and traceable way. It also creates metadata needed for later separation, result viewing, and history.
Frequency of Use	Occurs whenever the User wants to process a new mixed cardiopulmonary audio file during testing, demonstration, or review.
Triggers	The User selects a WAV file in the upload form and submits the upload request.
Preconditions	The web interface is available; the User has a mixed WAV audio file; local upload storage and SQLite are available.
Postconditions	A valid upload is saved in local storage, and a corresponding uploaded audio record is stored in SQLite with file name, path, timestamp, and metadata where available.
Main Course	**1.** The User opens the web interface. **2.** The User selects a mixed WAV audio file. **3.** The User submits the upload form. **4.** The system validates the file type and audio properties. **5.** The system stores the file in the upload folder. **6.** The system records upload metadata in SQLite. **7.** The system returns the uploaded audio identifier for separation.
Alternate Course	The User may upload another mixed WAV file after a successful upload. The system treats the second file as a new upload and creates a separate database record.
Exception Courses	If the file is missing, not a WAV file, unreadable, or unsupported, the system rejects the upload and shows a clear error message. If storage or database writing fails, the system does not continue to separation and records the failure where possible.

Table 2.2: RML-Style Use Case Specification for UC_002

Field	**Specification**
Name	Run Sound Separation
ID	`UC_002`
Description	The User starts cardiopulmonary sound separation for an uploaded mixed WAV file. The system uses the selected model if a model is chosen, or the active default model if no model is selected. NeoSSNet is the current real available separation model.
Actor	User
Organization Benefits	Demonstrates the main HealthTech function of the prototype by converting a mixed cardiopulmonary input into separate heart and lung sound outputs using real machine learning inference.
Frequency of Use	Occurs after a successful upload whenever the User wants to generate separated heart and lung outputs.
Triggers	The User clicks the separation action for an uploaded audio record and may choose an available model from the model dropdown.
Preconditions	A valid uploaded audio record exists; the uploaded WAV file is available in local storage; SQLite is available; the selected or active model record exists; the NeoSSNet checkpoint and configuration files are available.
Postconditions	A separation job record is created or updated, heart and lung WAV output files are stored locally, result metadata is saved in SQLite, and the job status shows completion or failure.
Main Course	**1.** The User selects or accepts the default separation model. **2.** The User starts separation for an uploaded audio file. **3.** The system retrieves the upload record and model record from SQLite. **4.** The system creates a running separation job. **5.** The system runs the selected or default separation model. **6.** NeoSSNet generates heart and lung WAV output files. **7.** The system stores output paths and job status in SQLite. **8.** The system returns the completed job information to the web interface.
Alternate Course	If the User does not choose a model, the system uses the active default model from SQLite. In the current prototype, this default model is NeoSSNet.
Exception Courses	If the uploaded file is missing, the selected model does not exist, the checkpoint path is unavailable, or inference fails, the system marks the job as failed and stores an error message. The system does not create fake heart or lung outputs.

Table 2.3: RML-Style Use Case Specification for UC_003

Field	**Specification**
Name	Preview Separated Audio
ID	`UC_003`
Description	The User previews the separated heart and lung WAV outputs in the browser after a successful separation job.
Actor	User
Organization Benefits	Allows quick inspection of separation results during demonstration or testing without requiring external audio software. This improves usability and makes the prototype easier to evaluate.
Frequency of Use	Occurs after separation completes successfully and whenever the User reopens a previous result from history.
Triggers	The system displays a completed result, or the User opens a completed job from processing history.
Preconditions	A separation job has completed successfully; result records exist in SQLite; heart and lung output files exist in local storage; the browser supports playback of the generated WAV files.
Postconditions	The User can listen to the separated heart and lung outputs through browser audio controls. No database record is changed by playback alone.
Main Course	**1.** The User opens the result view for a completed job. **2.** The system loads the stored heart and lung output paths from SQLite. **3.** The system displays browser audio controls for the separated outputs. **4.** The User plays the heart sound output. **5.** The User plays the lung sound output. **6.** The User compares the separated outputs with the original mixed audio if the original preview is available.
Alternate Course	The User may preview only one output, such as the heart sound, and still proceed to download or history without playing the other output.
Exception Courses	If an output file is missing, the system should avoid showing a broken playback experience and should display a clear unavailable result state. If the browser cannot play the file, the download option may still be used for external playback.

Table 2.4: RML-Style Use Case Specification for UC_004

Field	**Specification**
Name	Download Separated Audio
ID	`UC_004`
Description	The User downloads the generated heart or lung WAV output file from a completed separation job.
Actor	User
Organization Benefits	Supports offline review, presentation, comparison, and further analysis of generated separation outputs while keeping the runtime application simple and local.
Frequency of Use	Occurs whenever the User wants to keep or share a separated output after previewing or viewing a completed job.
Triggers	The User selects the download action for the heart output or lung output.
Preconditions	A completed separation result exists; the selected output file path is stored in SQLite; the requested heart or lung WAV file exists in local output storage.
Postconditions	The selected output file is delivered to the browser as a downloadable WAV file. The stored result record remains unchanged.
Main Course	**1.** The User opens a completed job result. **2.** The User selects either the heart output download or lung output download. **3.** The system checks the related result record. **4.** The system locates the requested WAV file in local storage. **5.** The system sends the file to the browser for download. **6.** The User saves or opens the downloaded file.
Alternate Course	The User may download both heart and lung outputs from the same completed job. Each download is handled as a separate file request.
Exception Courses	If the job ID is invalid, the output type is not recognised, or the requested file is missing from local storage, the system returns a clear error response instead of an empty or incorrect file.

Table 2.5: RML-Style Use Case Specification for UC_005

Field	**Specification**
Name	View Processing History
ID	`UC_005`
Description	The User views saved processing history from SQLite, including previous uploads, separation jobs, model information, statuses, result links, and timestamps.
Actor	User
Organization Benefits	Provides traceability for the prototype by showing how uploaded files, selected models, generated outputs, and job statuses are connected. This supports demonstration, debugging, and report explanation.
Frequency of Use	Occurs whenever the User wants to review previous processing activity or reopen a completed result.
Triggers	The User opens the history section or refreshes the main web interface history view.
Preconditions	The FastAPI backend and SQLite database are available. History may contain previous records, or it may be empty in a fresh setup.
Postconditions	The web interface displays available processing history records, including completed, failed, or pending jobs where applicable.
Main Course	**1.** The User opens the processing history section. **2.** The system queries saved upload, job, model, and result records from SQLite. **3.** The system formats the records for the web interface. **4.** The system displays job status, original file information, model used, timestamps, and result actions where available. **5.** The User reviews previous activity or opens a completed result.
Alternate Course	If no previous jobs exist, the system displays an empty history state and keeps the upload workflow available.
Exception Courses	If the database cannot be read or a history record points to a missing output file, the system should display the available metadata with a clear error state for the affected result instead of failing the whole page.

2.2 Class Diagram

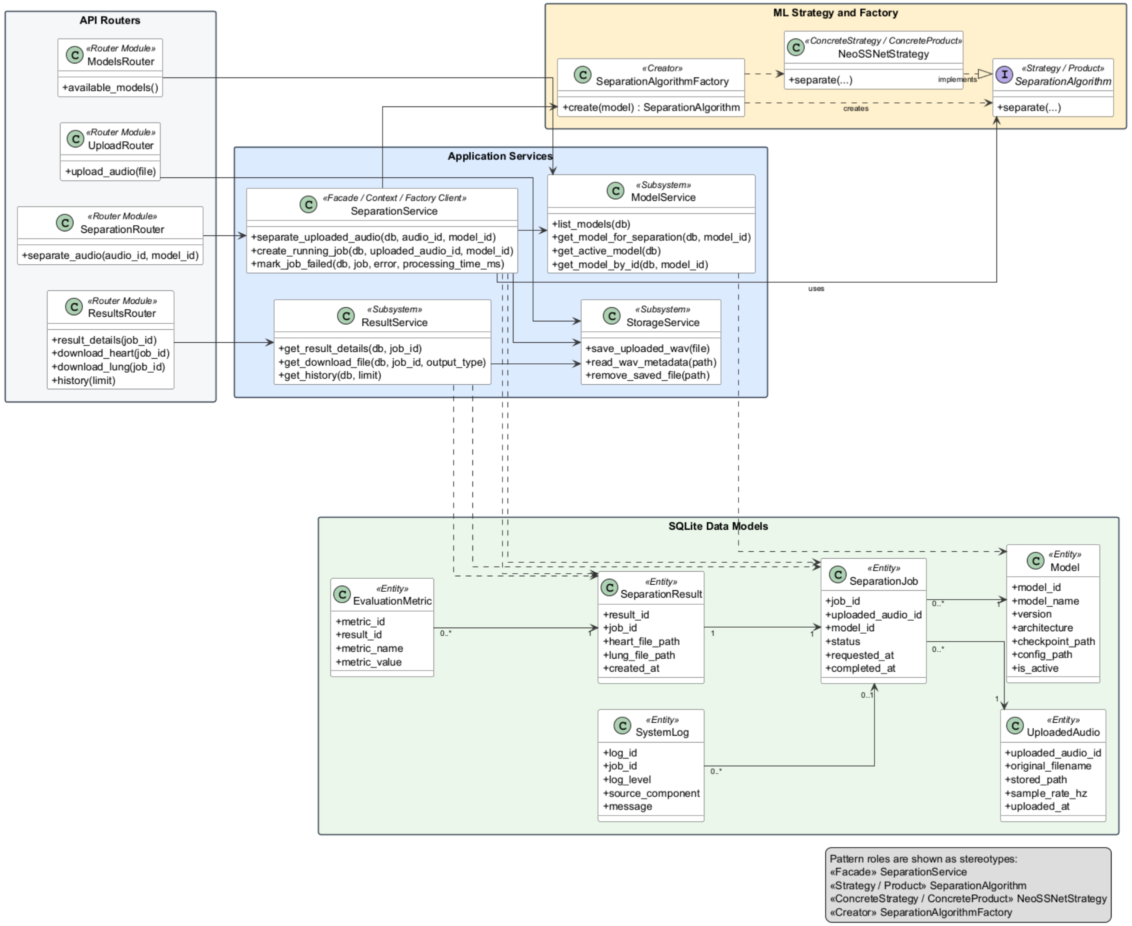

Figure 2.2: Overall System UML Class Diagram

Figure 2.2 presents the main classes and interfaces in the cardiopulmonary sound separation system. Each class or interface appears only once, and the diagram shows the API router modules, service components, machine learning strategy and factory classes, and SQLite data model classes. The diagram uses UML class-style boxes, selected attributes or methods, associations, dependencies, and interface implementation arrows to describe the whole system structure.

Design pattern roles are shown through stereotypes and the diagram legend rather than by duplicating class boxes. For example, `SeparationService` is marked as the Facade, Strategy Context, and Factory Client; `SeparationAlgorithm` is marked as the Strategy/Product abstraction; `NeoSSNetStrategy` is marked as the Concrete Strategy/Concrete Product; and `SeparationAlgorithmFactory` is marked as the Creator. This keeps the official class diagram correct while still making the selected design patterns visible.

2.3 Entity Relationship Diagram (ERD)

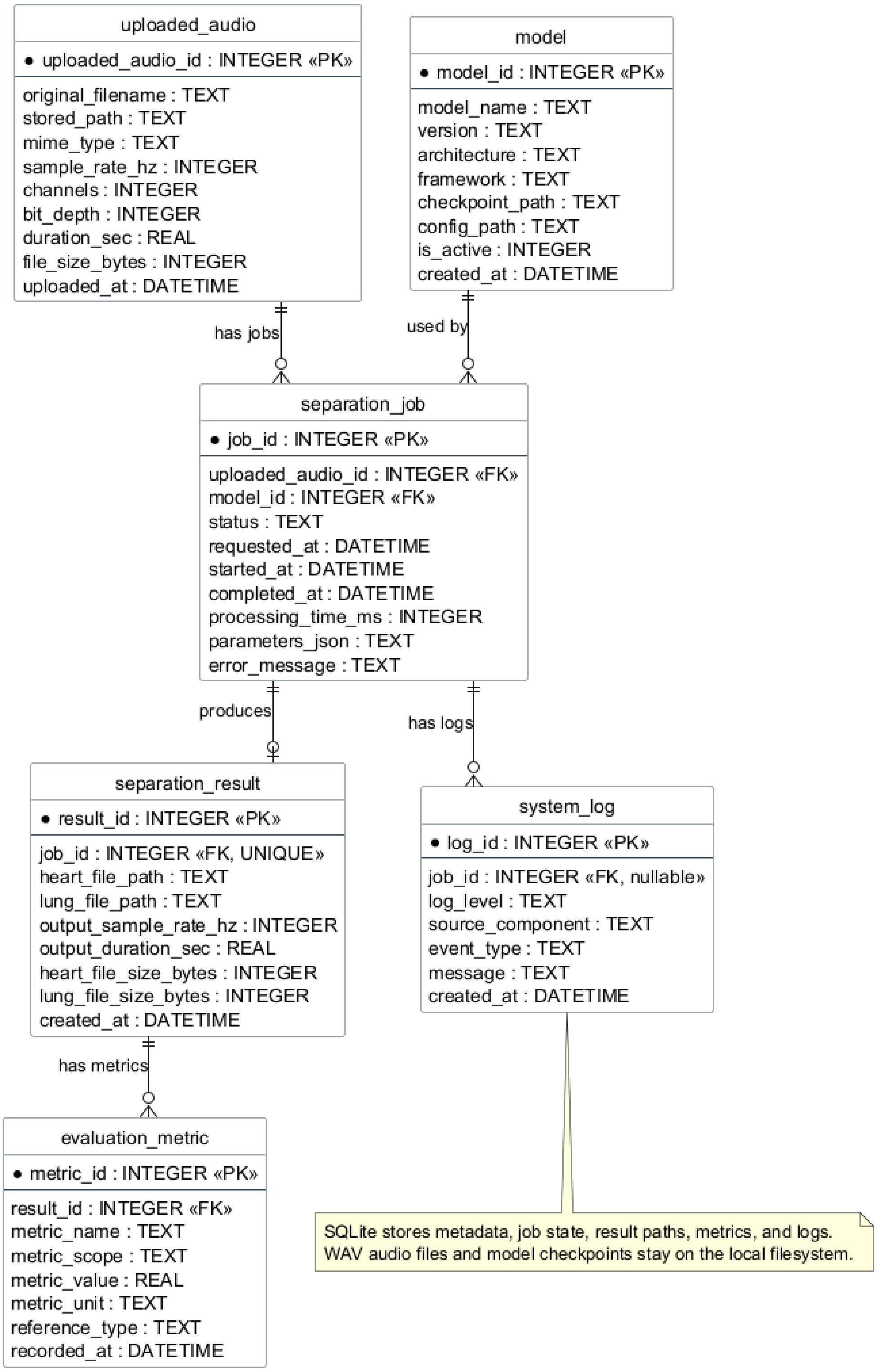

Figure 2.3: Entity Relationship Diagram

Figure 2.3 shows the SQLite database structure used to store structured project data. The `uploaded_audio` table records each uploaded WAV file, the `model` table records available model information, and the `separation_job` table connects an uploaded file with the model used for processing. The `separation_result` table stores the paths and metadata for generated heart and lung outputs.

The design intentionally stores file paths and metadata in SQLite rather than storing large WAV audio blobs in the database. This supports a practical student project architecture: local storage keeps uploaded and generated audio files, while SQLite supports history, result lookup, job status, metrics, and logs. As a result, the browser can display previous jobs and download links without scanning folders manually.

2.4 Object Diagram

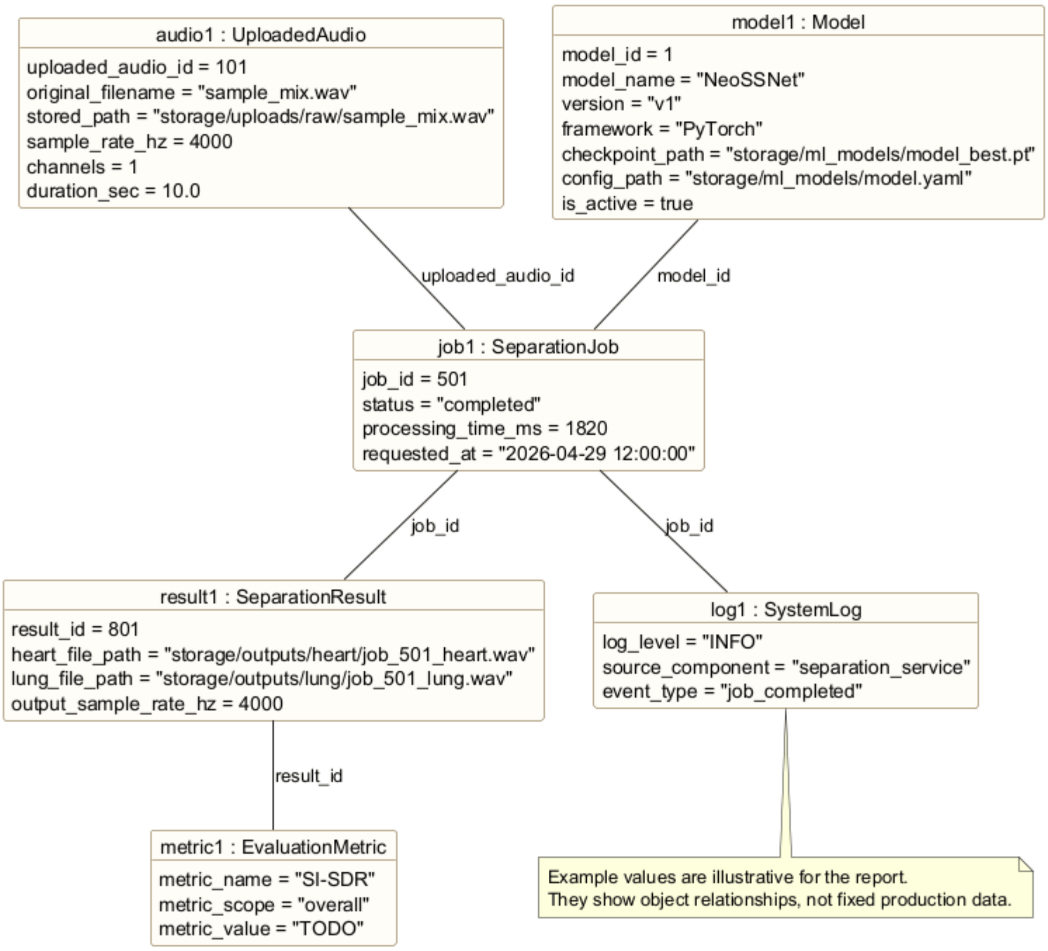

Figure 2.4: Object Diagram for a Completed Separation Job

Figure 2.4 gives a concrete example of how the system data may look after one successful separation. One `UploadedAudio` object is linked to one `SeparationJob`, the job uses an active `Model`, and the completed job produces a `SeparationResult` containing heart and lung output file paths.

The object diagram helps explain how upload, separation, storage, result viewing, and history are connected at runtime. When a user views history, the system can retrieve the job, original upload metadata, model information, output paths, optional metrics, and log record as related objects. This gives the project a traceable flow from the original mixed input to the separated outputs.

Requirements

This chapter defines the main functional requirements, non-functional requirements, software design concepts, and design principles for the cardiopulmonary sound separation system. The system is treated as a prototype and proof of concept for a software design assignment and Final Year Project context. It is designed to demonstrate a realistic local web application that can upload mixed cardiopulmonary audio, run NeoSSNet-based separation, store metadata, and present results clearly to students, lecturers, researchers, or demonstration users. The requirements do not claim clinical validation; they focus on software behaviour, maintainability, and demonstration readiness.

3.1 Functional Requirements

Functional requirements describe the main behaviours that the system must provide to the user and to the backend workflow.

Table 3.1: Functional Requirements

ID	Requirement	Description
FR-01	Upload mixed WAV audio	The user shall be able to select and upload a mixed cardiopulmonary WAV file through the web interface.
FR-02	Validate audio file	The backend shall check that the uploaded file is a valid supported audio file before saving it for processing.
FR-03	Run cardiopulmonary sound separation	The backend shall run the separation workflow using NeoSSNet to separate the mixed input into heart and lung sound outputs.
FR-04	Generate heart and lung output files	The system shall save separated heart and lung audio files in the configured output folders.
FR-05	Preview separated audio	The web interface shall allow the user to preview the original audio and the separated heart and lung outputs where browser playback is supported.
FR-06	Download separated audio	The user shall be able to download the generated heart and lung output files for later review or demonstration.
FR-07	View processing history	The system shall show previous uploads, jobs, statuses, and result links so that users can review earlier processing activity.
FR-08	Store logs and metadata	The system shall store upload metadata, job records, result paths, metrics where available, and important processing logs in SQLite.

These functional requirements support the main demonstration flow of the project. A user uploads a mixed sound, the backend validates and processes it, the system stores the generated files, and the browser presents previews, downloads, and history. This keeps the project focused on a working end-to-end software system rather than only a standalone machine learning script.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes expected from the prototype. These qualities are important because the system should be understandable, maintainable, and reliable enough for local testing and presentation.

Table 3.2: Non-Functional Requirements

Quality Attribute	**Requirement**
Usability	The interface should make the upload, separation, preview, download, and history flow clear for a student or examiner during a demonstration.
Maintainability	The code should be organised into clear frontend, router, service, machine learning, database, and storage responsibilities so that future changes are easier to make.
Reliability	The backend should handle invalid files, failed separation jobs, missing outputs, and database errors with clear status messages and logs.
Modularity	Each major part of the system should be replaceable or testable with minimal impact on unrelated components.
Performance	The system should process short demonstration WAV files within a reasonable time on a local machine, while avoiding unnecessary database storage of large audio blobs.
Data integrity	SQLite records should consistently link uploaded audio, model records, separation jobs, results, metrics, and logs. Failed jobs should keep useful error information.
Portability and local execution	The system should run locally using Python, FastAPI, SQLite, PyTorch, and local folders without requiring cloud services or complex deployment infrastructure.

The non-functional requirements guide implementation decisions. For example, local execution supports a practical student project setup, while data integrity and logging support traceability when explaining how a mixed input becomes separated heart and lung outputs.

3.3 Software Design Concepts

3.3.1 Abstraction

Abstraction is used to hide detailed processing steps behind simpler operations. In this project, the separation route does not need to know how model records are selected, how NeoSSNet is called, or how heart and lung output paths are written. It can call a service-level operation such as `separate_uploaded_audio` and treat the workflow as one meaningful software action.

The machine learning boundary also uses abstraction. `SeparationAlgorithm` represents the expected behaviour of a separation model, while `NeoSSNetStrategy` provides the current real implementation. This allows the service layer to express the workflow in terms of cardiopulmonary sound separation rather than in terms of one concrete model class.

3.3.2 Modularity

Modularity divides the system into smaller parts with clear responsibilities. The frontend handles upload controls, audio preview, download links, and history display. FastAPI routers handle HTTP requests. Services coordinate workflows. The ML layer performs real separation. SQLite models store structured records, and local folders store uploaded and generated WAV files.

This modular structure is useful for the prototype because a change in one area does not require rewriting the entire system. For example, improving the frontend model dropdown should not require changing NeoSSNet inference, and changing output folder naming should not require changing the database schema.

3.3.3 Separation of Concerns

Separation of concerns prevents unrelated responsibilities from being mixed together. The upload route receives files, the storage service manages local file placement, the model service reads available model records, the separation service coordinates processing, and the ML strategy runs inference. This keeps web request handling, file management, model selection, and audio separation as separate concerns.

The same idea applies to data storage. Runtime upload folders, generated output folders, model checkpoints, dataset folders, and SQLite metadata serve different purposes and remain separate. This design avoids training from runtime upload folders and avoids storing large WAV content directly in SQLite.

3.3.4 Information Hiding

Information hiding means that internal implementation details are kept inside the module or class responsible for them. File validation and path rules should stay inside upload or storage-related code. Model selection rules should stay inside `ModelService`. Algorithm construction should stay inside `SeparationAlgorithmFactory`. NeoSSNet inference details should stay inside the ML strategy and inference function.

This protects the rest of the system from unnecessary implementation knowledge. For example, the route does not need to know where the checkpoint file is stored or how the strategy calls the underlying inference function; it only needs the job result returned by the service.

3.3.5 Low Coupling and High Cohesion

Low coupling means that components should depend on each other only where necessary. In this project, `SeparationService` depends on the `SeparationAlgorithm` abstraction rather than depending directly on NeoSSNet-specific workflow logic. This reduces the impact of future model changes because the upload, result, history, and download routes can remain stable.

High cohesion means that each component should focus on related tasks. `StorageService` is cohesive when it handles file storage concerns, `ModelService` is cohesive when it handles model lookup, and `NeoSSNetStrategy` is cohesive when it only adapts NeoSSNet inference to the shared algorithm interface. Cohesion helps the system remain easy to explain and test.

3.4 Design Principles

3.4.1 Single Responsibility Principle

The Single Responsibility Principle supports the assignment of one main reason for change to each component. For example, `StorageService` should change when file storage rules change, not when separation model logic changes. This keeps file handling separate from ML inference and database workflow decisions.

Listing 3.1: Single Responsibility Principle Example

```python
# Single Responsibility Principle:
# StorageService handles storage concerns only.
class StorageService:
    def save_uploaded_wav(self, upload_file) -> str:
        stored_path = build_upload_path(upload_file.filename)
        write_file(upload_file, stored_path)
        return stored_path


# Request handling remains outside StorageService.
def upload_audio_route(file, db):
    stored_path = StorageService().save_uploaded_wav(file)
    return create_uploaded_audio_record(db, file.filename, stored_path)
```

In this example, the storage class does not create separation jobs and does not run NeoSSNet.

Its responsibility is limited to saving files and returning a path that other components can use.

3.4.2 Open/Closed Principle

The Open/Closed Principle suggests that the system should be open for extension but closed for unnecessary modification. The separation workflow should be able to support a new implemented algorithm by adding a strategy class and registering it in the factory, without rewriting upload, result, history, or download routes.

Listing 3.2: Open/Closed Principle Example

```
# Open for extension:
# add a new implemented strategy and register it in the factory.
class NeoSSNetStrategy:
    def separate(self, input_wav_path, model_path, config_path,
                 heart_output_path, lung_output_path):
        return run_neossnet_inference(...)


class SeparationAlgorithmFactory:
    _registry = {
        "neossnet": NeoSSNetStrategy,
    }

    @classmethod
    def create(cls, model):
        strategy_class = cls._registry[model.architecture.lower()]
        return strategy_class()
```

The workflow is closed to unnecessary modification because `SeparationService` can keep the same high-level steps. A future algorithm should mainly require a new concrete strategy and a factory registration.

3.4.3 Dependency Inversion Principle

Dependency Inversion is relevant because high-level workflow classes should not be tightly bound to one concrete model implementation. `SeparationService` should depend on the `SeparationAlgorithm` abstraction, while `NeoSSNetStrategy` provides the current concrete implementation.

Listing 3.3: Dependency Inversion Principle Example

```
# Dependency Inversion Principle:
# high-level workflow depends on the abstraction.
class SeparationService:
    def run_algorithm(self, algorithm: SeparationAlgorithm, paths):
        return algorithm.separate(
            input_wav_path=paths.input_wav,
            model_path=paths.model_checkpoint,
            model_config_path=paths.model_config,
            heart_output_path=paths.heart_output,
            lung_output_path=paths.lung_output,
        )
```

The service does not need to instantiate or know the internal details of `NeoSSNetStrategy`. The factory supplies an object that satisfies the `SeparationAlgorithm` interface, and the service runs the workflow through that abstraction.

3.4.4 Interface-Based Design for Algorithm Replacement

Interface-based design supports future algorithm replacement by defining the expected input and output of a separation algorithm. The current algorithm should produce separated heart and lung audio outputs, but the service layer should not need to know whether the implementation uses NeoSSNet, an ONNX-exported model, or a future baseline method. For the current project, NeoSSNet remains the required real separation model.

3.4.5 Persistence Separation Between SQLite and Filesystem

The design separates structured metadata from large binary audio files. SQLite stores records such as filenames, file paths, upload timestamps, job status, model information, output paths, metrics, and logs. The filesystem stores uploaded WAV files, generated heart and lung outputs, model checkpoints, and dataset files. This approach is simpler to run locally and avoids making the database unnecessarily large.

3.5 Software Design Plan

The proposed software design is organised into layers. This layered plan supports a clear implementation path and makes the system easier to explain in the report and viva presentation.

Table 3.3: Software Design Plan by Layer

Layer	Main Responsibility	Project Role
Frontend layer	HTML, CSS, JavaScript, templates, and browser audio controls	Provides upload form, status messages, original and separated audio playback, download links, and history display.
Backend API layer	FastAPI application and routers	Receives upload, separation, result, download, history, log, and health requests.
Service layer	Workflow coordination and business logic	Connects validation, database updates, NeoSSNet inference, output storage, logging, and result retrieval.
ML inference layer	Audio preprocessing and model inference	Loads or prepares audio, applies the NeoSSNet model, and returns separated heart and lung waveforms.
Database layer	SQLite schema and repository-style access	Stores uploaded audio metadata, model records, job states, result paths, metrics, and system logs.
Storage layer	Local filesystem folders	Stores uploaded mixed WAV files, temporary files, separated heart and lung outputs, logs where needed, and ML model weights.

The design plan follows the project priority of real functionality first, then simplicity and local execution. It avoids unnecessary cloud, microservice, or authentication complexity unless those features are explicitly added later.

Proposed Design Patterns

This chapter proposes three design patterns for the cardiopulmonary sound separation system. The patterns are presented as practical backend design structures for the FastAPI, SQLite, local storage, and NeoSSNet pipeline. Each pattern is linked to a specific project problem so that the design remains useful for implementation and explanation during the final presentation.

The class and method names used in this chapter are aligned with the current model-selection implementation. The report uses `SeparationService` to describe the service role implemented in `app/services/separation_service.py`, while the actual code also includes `SeparationAlgorithm`, `NeoSSNetStrategy`, `SeparationAlgorithmFactory`, and `ModelService`.

4.1 Facade Pattern

The Facade Pattern is proposed to provide a simple interface over the separation workflow, which contains several lower-level operations such as audio preprocessing, model inference, file storage, database updates, and logging (Gamma et al., 1994).

4.1.1 Pattern Type

The Facade Pattern is a structural design pattern. It focuses on arranging classes so that a client can access a complex subsystem through a simpler and more stable interface.

4.1.2 Problem

The separation endpoint should not directly coordinate every step of the backend workflow. If the FastAPI router directly calls the audio preprocessor, NeoSSNet inference class, storage utilities, database session, and logging logic, the route becomes difficult to read, test, and modify. This would also make the design harder to explain because web routing logic and machine learning workflow logic would be mixed in one place.

4.1.3 Solution

The solution is to use `SeparationService` as the facade for the separation workflow. The router calls one high-level operation, `separate_uploaded_audio(db, audio_id, model_id)`, while the service coordinates model selection, algorithm creation, job creation, inference, output storage, and database result updates. This keeps the router focused on request handling and keeps the separation workflow inside a dedicated service layer.

4.1.4 Structure

The template structure below shows how a facade sits between a client and a group of subsystem classes. The client depends on the facade instead of depending on each subsystem class directly.

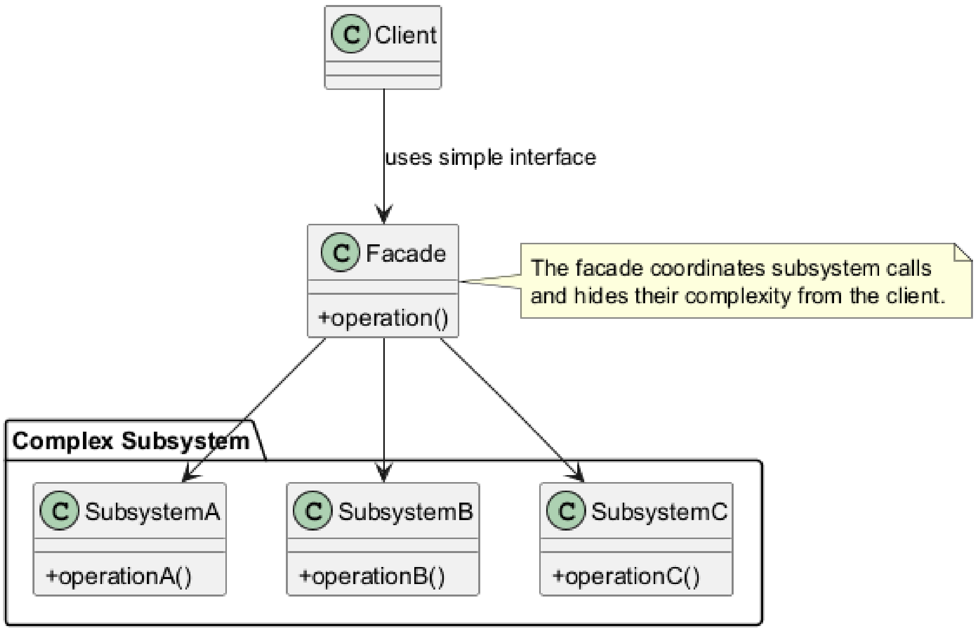

Figure 4.1: Facade Pattern Template Structure

1. **Client:** The external caller that needs to use the subsystem. In this project, the client is mainly the FastAPI router that receives a separation request.
2. **Facade:** The simplified interface used by the client. In this project, `SeparationService` hides the detailed separation steps behind one service method.
3. **Complex Subsystem:** The lower-level components that perform the actual work. In this project, these include `ModelService`, `SeparationAlgorithmFactory`, `SeparationAlgorithm`, `NeoSSNetStrategy`, SQLite database records, and local file storage.

4.1.5 Project Application

i. Function or Software Component Affected

The affected component is the backend separation workflow, especially the service layer that connects the FastAPI separation route to model selection, machine learning inference, SQLite records, and local file storage. The main facade role is `SeparationService`, represented by the `separate_uploaded_audio` workflow in `app/services/separation_service.py`.

ii. Description of Workflow or Data Flow

The router receives a request to separate an uploaded WAV file and may also receive an optional `model_id`. It passes the database session, uploaded audio identifier, and selected model identifier to `SeparationService`. The service loads the upload metadata from SQLite, selects either the requested model or the active model through `ModelService`, creates the correct algorithm through `SeparationAlgorithmFactory`, runs the selected `SeparationAlgorithm`, saves separated heart and lung WAV files to local storage, and records the job and result metadata in SQLite.

iii. Sample Class Diagram

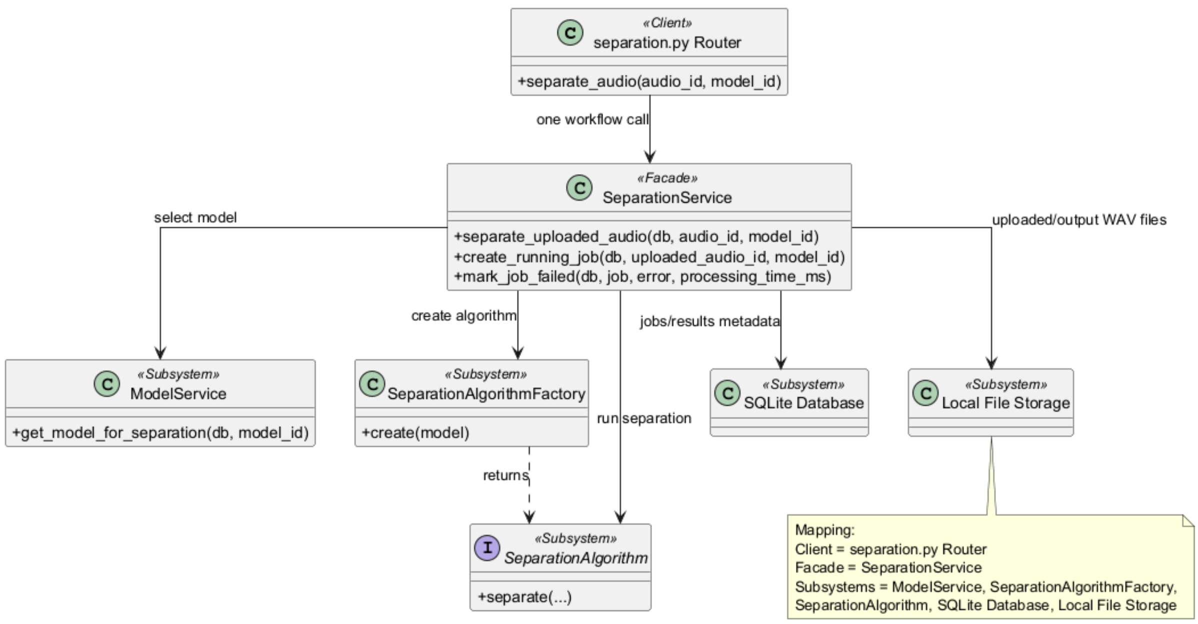

Figure 4.2: Facade Pattern Project Class Diagram

iv. Facade Pattern Sequence Diagram

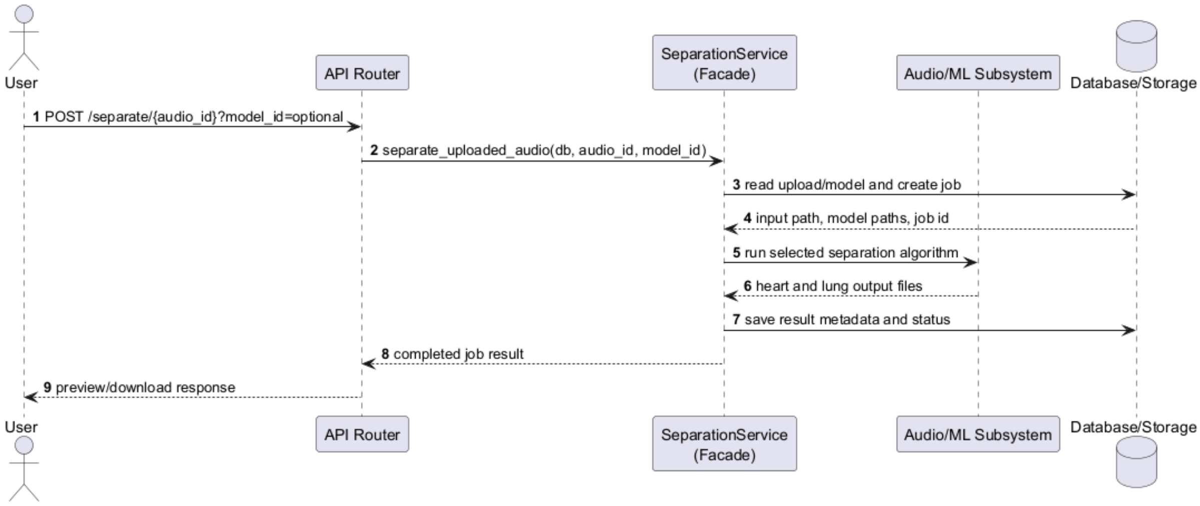

Figure 4.3: Facade Pattern Sequence Diagram

v. Sample Potential Code

Listing 4.1: Facade Pattern Code Example

```
# Client: FastAPI route in app/routers/separation.py
def separate_audio(audio_id: int, model_id: int | None, db):
    service = SeparationService()
    return service.separate_uploaded_audio(db, audio_id, model_id)


# Subsystem: model lookup
class ModelService:
    def get_model_for_separation(self, db, model_id=None):
        return get_requested_or_active_model(db, model_id)


# Subsystem: algorithm creation
class SeparationAlgorithmFactory:
    def create(self, model):
        return NeoSSNetStrategy()


# Subsystem: selected ML algorithm
class NeoSSNetStrategy:
    def separate(self, input_wav_path, model_path, model_config_path,
```

```
                  heart_output_path, lung_output_path):
        return run_neossnet_inference(...)


# Facade: app/services/separation_service.py
class SeparationService:
    def separate_uploaded_audio(self, db, audio_id, model_id=None):
        upload = get_uploaded_audio(db, audio_id)
        model = ModelService().get_model_for_separation(db, model_id)
        algorithm = SeparationAlgorithmFactory().create(model)

        job = create_running_job(db, upload.uploaded_audio_id, model.model_id)
        output_paths = build_output_paths(job.job_id)

        result = algorithm.separate(
            input_wav_path=upload.stored_path,
            model_path=model.checkpoint_path,
            model_config_path=model.config_path,
            heart_output_path=output_paths.heart,
            lung_output_path=output_paths.lung,
        )

        save_result_metadata(db, job, result)
        return job.job_id
```

The code sample is simplified for readability, but the roles and method names follow the implemented backend structure.

vi. Benefits

The facade makes the route simpler, improves separation of concerns, and makes the workflow easier to test as one service operation. It also improves presentation clarity because the main separation process can be explained as a single service call that coordinates several internal components.

vii. Limitations

The main limitation is that the facade can become too large if every processing rule is placed inside `SeparationService`. The implementation should keep detailed work inside subsystem classes and use the facade mainly for orchestration.

4.2 Strategy Pattern

The Strategy Pattern is used to keep the separation algorithm interchangeable behind a common interface. This is useful because the project currently uses NeoSSNet, but the backend should still allow another implemented algorithm to be added later without changing the upload, result, history, or download routes (Gamma et al., 1994).

4.2.1 Pattern Type

The Strategy Pattern is a behavioral design pattern. It focuses on defining a family of algorithms and allowing the context object to use them through a shared interface.

4.2.2 Problem

The system should not hardcode NeoSSNet-specific logic throughout the service layer. If model-specific preprocessing, inference, and output handling are tightly coupled to the route or service, future experiments become harder. Even for a student project, keeping the algorithm boundary clear makes the design easier to test and explain.

4.2.3 Solution

The solution is to define a `SeparationAlgorithm` interface with a `separate(...)` operation. `NeoSSNetStrategy` is the current concrete strategy and wraps the existing real `run_neossnet_inference` function. `SeparationService` acts as the context and calls the selected strategy through the shared interface.

4.2.4 Structure

The template structure below shows a context object that delegates algorithm-specific work to a strategy object. The client uses the context without needing to know which concrete strategy performs the work.

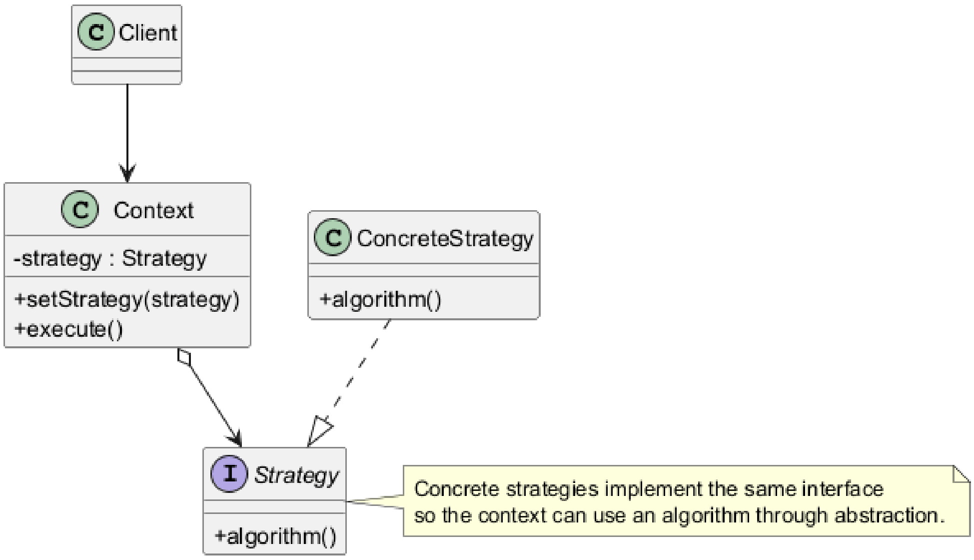

Figure 4.4: Strategy Pattern Template Structure

1. **Context:** The object that uses a strategy to complete part of its work. In this project, `SeparationService` is the context because it delegates model-specific separation to an algorithm object.

2. **Strategy Interface:** The common contract for all algorithms. In this project, `SeparationAlgorithm` defines the expected separation method and output format.

3. **Concrete Strategies:** The actual algorithm implementations. In the current implementation, `NeoSSNetStrategy` is the concrete strategy.

4. **Client:** The caller that starts the workflow. In this project, the FastAPI router starts the separation request, while the service uses the configured strategy internally.

4.2.5 Project Application

i. Function or Software Component Affected

The affected component is the machine learning inference boundary. The main classes are `SeparationAlgorithm`, `NeoSSNetStrategy`, and `SeparationService`. The interface is intentionally small so that a new implemented strategy can be registered later without changing the separation route or result workflow.

ii. Description of Workflow or Data Flow

The service resolves the uploaded WAV path and model paths, then calls the selected `SeparationAlgorithm`. The current concrete strategy, `NeoSSNetStrategy`, delegates to the existing NeoSSNet inference function and returns a `SeparationAlgorithmResult` containing heart output path, lung output path, sample rate, duration, and file size metadata. The service records these outputs without depending on NeoSSNet internals.

iii. Sample Class Diagram

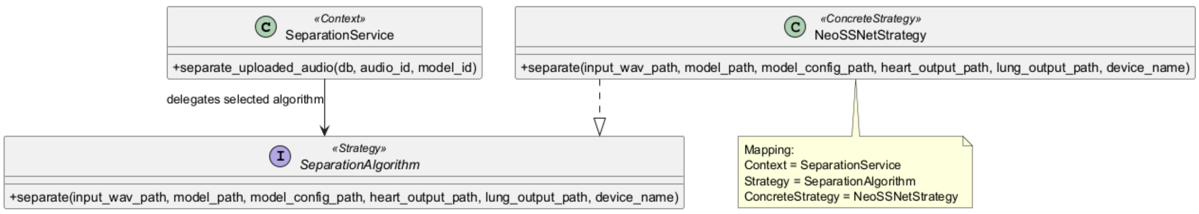

Figure 4.5: Strategy Pattern Project Class Diagram

iv. Strategy Pattern Sequence Diagram

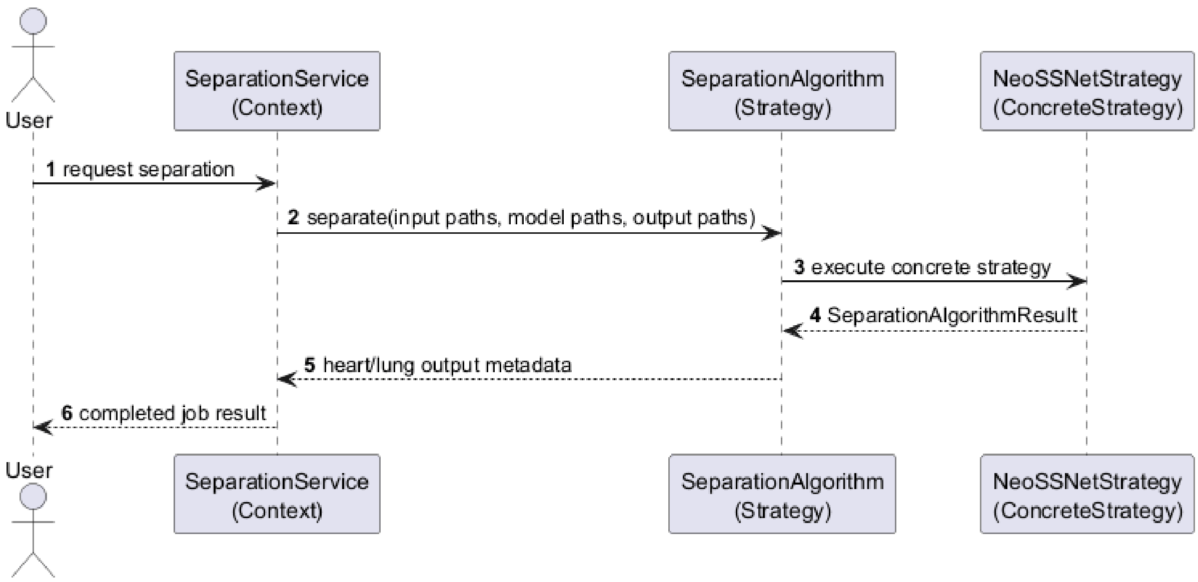

Figure 4.6: Strategy Pattern Sequence Diagram

v. Sample Potential Code

Listing 4.2: Strategy Pattern Code Example

```python
# Strategy: app/ml/separation_algorithm.py
class SeparationAlgorithm(Protocol):
    def separate(
        self,
        input_wav_path,
        model_path,
        model_config_path,
        heart_output_path,
        lung_output_path,
        device_name="cpu",
    ) -> SeparationAlgorithmResult:
        ...


# ConcreteStrategy: app/ml/neossnet_strategy.py
class NeoSSNetStrategy:
    def separate(self, input_wav_path, model_path, model_config_path,
                 heart_output_path, lung_output_path, device_name="cpu"):
        return run_neossnet_inference(
            input_wav_path=input_wav_path,
            model_path=model_path,
            model_config_path=model_config_path,
            heart_output_path=heart_output_path,
            lung_output_path=lung_output_path,
            device_name=device_name,
        )


# Context: app/services/separation_service.py
class SeparationService:
    def run_selected_algorithm(
        self,
        algorithm: SeparationAlgorithm,
        input_path,
        model,
        output_paths,
    ):
        return algorithm.separate(
            input_wav_path=input_path,
            model_path=model.checkpoint_path,
            model_config_path=model.config_path,
```

```
        heart_output_path=output_paths.heart,
        lung_output_path=output_paths.lung,
    )
```

The concrete strategy does not duplicate NeoSSNet inference logic; it delegates to the existing inference function.

vi. Benefits

The strategy design makes the algorithm boundary clear, improves testability, and supports future comparison with alternative separation methods. It also helps keep NeoSSNet-specific code inside a dedicated class rather than spreading it across the router and service layer.

vii. Limitations

The limitation is that the project may only use NeoSSNet during the final submission. If no alternative algorithm is implemented, the extra interface should remain small so it does not make the student project unnecessarily complicated.

4.3 Factory Method Pattern

The Factory Method Pattern is proposed to centralise the creation of separation algorithm objects. This avoids scattering model creation, checkpoint loading, and strategy selection logic throughout the backend (Gamma et al., 1994).

4.3.1 Pattern Type

The Factory Method Pattern is a creational design pattern. It focuses on creating objects through a dedicated creator method instead of directly constructing concrete classes in client code.

4.3.2 Problem

The system stores model-related information such as model name, version, checkpoint path, and active status in SQLite or local configuration. If each service directly constructs

`NeoSSNetStrategy` and loads checkpoint paths manually, model creation becomes duplicated and harder to change. This is especially risky if the project later supports more than one model format.

4.3.3 Solution

The solution is to use `SeparationAlgorithmFactory` as the creator. The service obtains the selected or active model through `ModelService`, then passes that model record to the factory. The factory checks the model architecture and returns a `SeparationAlgorithm` object. At the current stage, the concrete product is `NeoSSNetStrategy`.

4.3.4 Structure

The template structure below shows a creator that returns a product interface while hiding the exact concrete product class from the client.

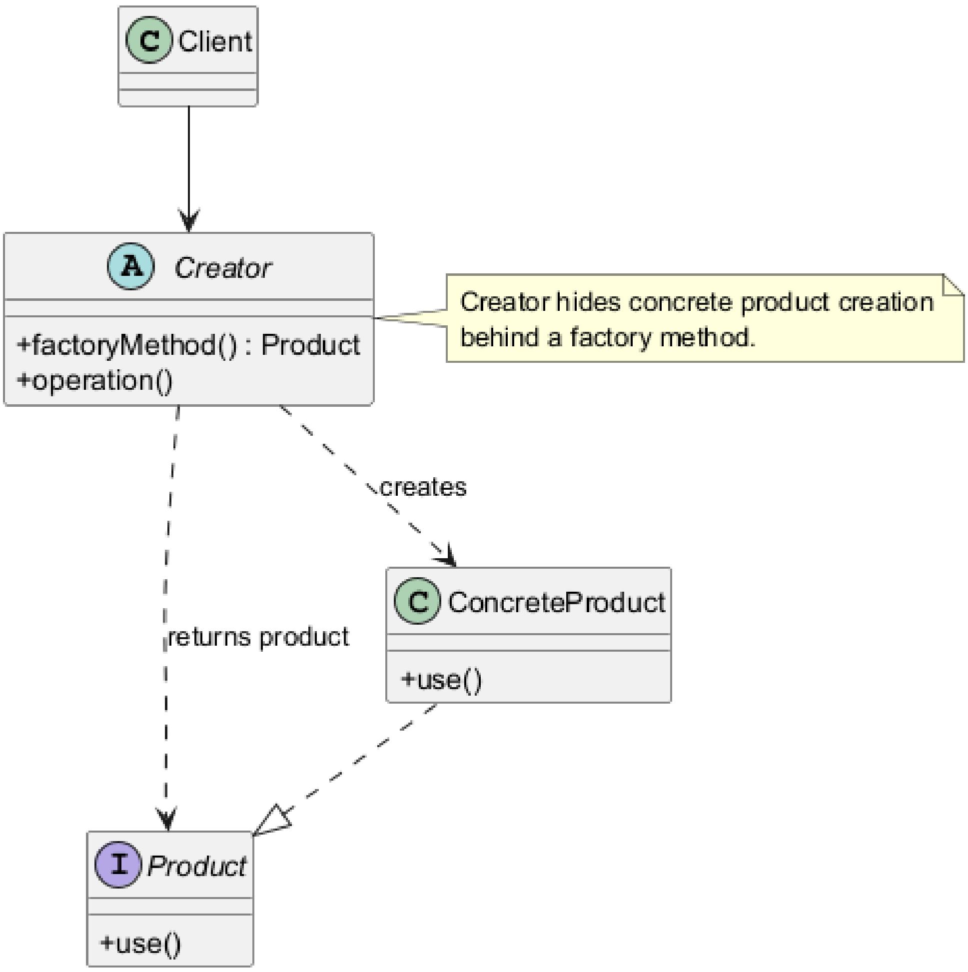

Figure 4.7: Factory Method Pattern Template Structure

1. **Creator:** The class or method responsible for creating product objects. In this project, `SeparationAlgorithmFactory` is the proposed creator.
2. **Product:** The shared interface returned to the client. In this project, `SeparationAlgorithm` is the product interface.
3. **Concrete Product:** The actual object created by the factory. In this project, `NeoSSNetStrategy` is the main concrete product.
4. **Client:** The class that requests a product from the factory. In this project, `SeparationService` uses the factory so that it does not need to hardcode model construction details.

4.3.5 Project Application

i. Function or Software Component Affected

The affected component is model and algorithm construction. The implemented creator is `SeparationAlgorithmFactory`, and it is used by `SeparationService`. Model lookup is handled separately by `ModelService`.

ii. Description of Workflow or Data Flow

The service requests the selected or active model from `ModelService`. It then passes the model record to `SeparationAlgorithmFactory.create(model)`. The factory checks `model.architecture`; when it is `NeoSSNet`, it creates `NeoSSNetStrategy`. The created object is returned through the `SeparationAlgorithm` interface, so the service can call `separate(...)` without hardcoding the concrete strategy construction.

iii. Sample Class Diagram

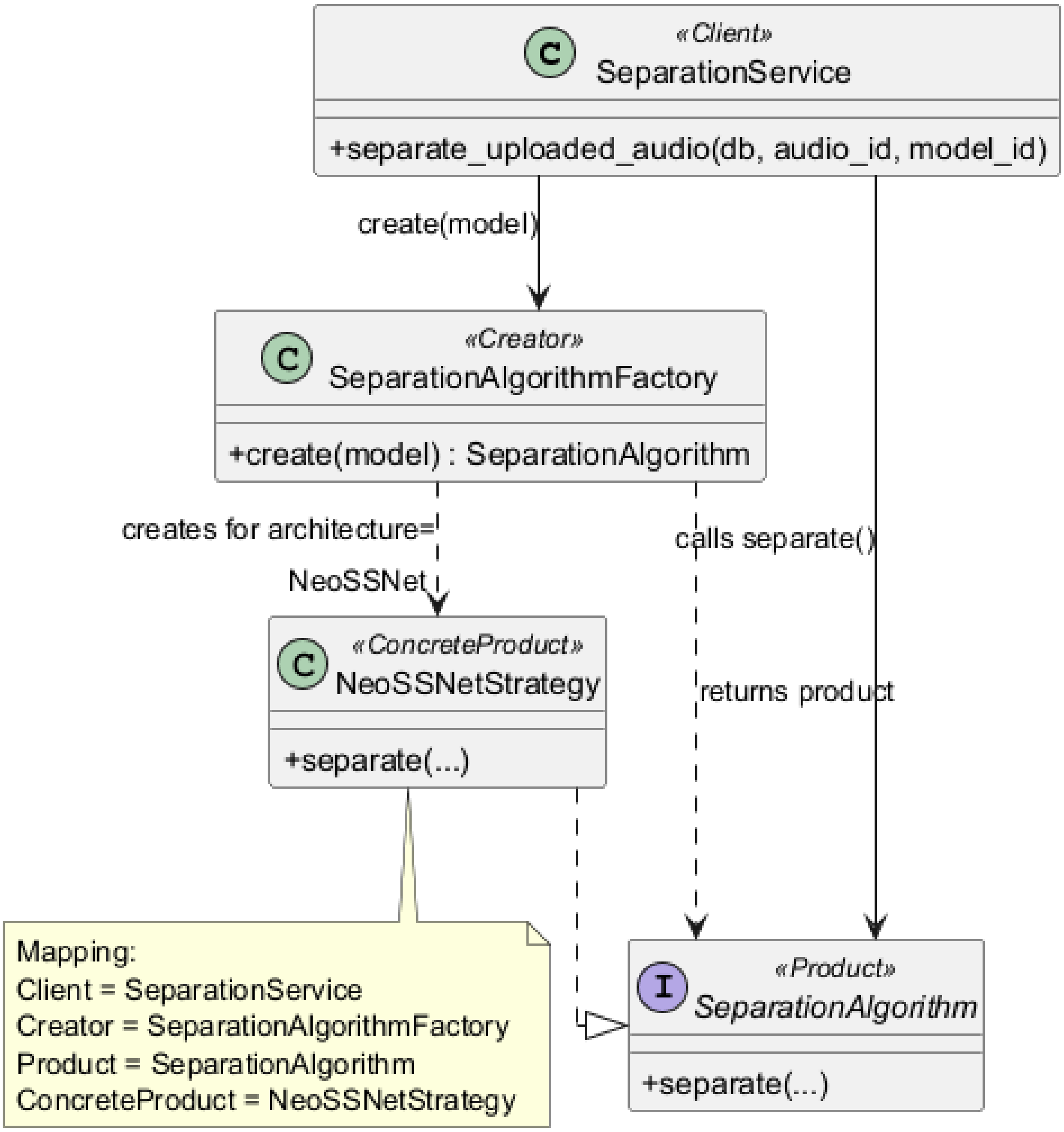

Figure 4.8: Factory Method Pattern Project Class Diagram

iv. Factory Method Pattern Sequence Diagram

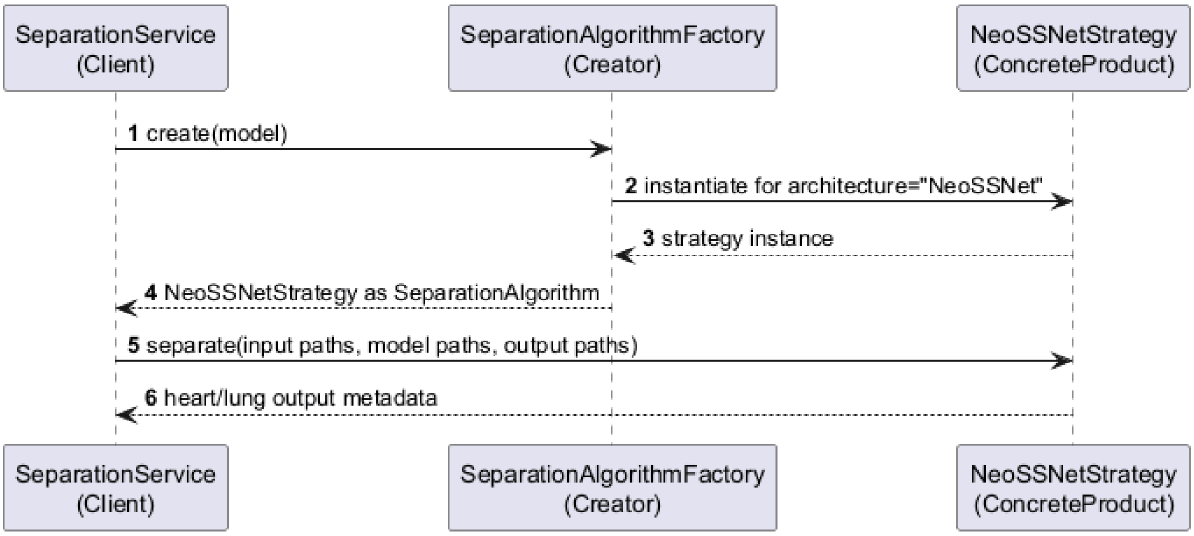

Figure 4.9: Factory Method Pattern Sequence Diagram

v. Sample Potential Code

Listing 4.3: Factory Method Pattern Code Example

```
# Product: app/ml/separation_algorithm.py
class SeparationAlgorithm(Protocol):
    def separate(self, input_wav_path, model_path, model_config_path,
                 heart_output_path, lung_output_path, device_name="cpu"):
        ...


# ConcreteProduct: app/ml/neossnet_strategy.py
class NeoSSNetStrategy:
    def separate(self, input_wav_path, model_path, model_config_path,
                 heart_output_path, lung_output_path, device_name="cpu"):
        return run_neossnet_inference(...)


# Creator: app/services/model_factory.py
class SeparationAlgorithmFactory:
    _registry = {
        "neossnet": NeoSSNetStrategy,
    }

    @classmethod
```

```python
    def create(cls, model) -> SeparationAlgorithm:
        architecture = model.architecture.strip().lower()
        strategy_class = cls._registry.get(architecture)
        if strategy_class is None:
            raise UnsupportedModelArchitectureError(architecture)

        return strategy_class()


# Client: app/services/separation_service.py
class SeparationService:
    def get_algorithm(self, db, model_id=None):
        model = ModelService.get_model_for_separation(db, model_id)
        return SeparationAlgorithmFactory.create(model)
```

The factory creates the strategy object only. The model checkpoint and configuration paths remain in the model record and are passed into the strategy during the separation call.

vi. Benefits

The factory method reduces hardcoded model creation, keeps checkpoint loading in one place, and makes it easier to add a future model type without changing every service that needs an algorithm. It also supports cleaner testing because a test can provide a simple model configuration and check which strategy is created.

vii. Limitations

The limitation is that a factory may be unnecessary if the system permanently supports only one model. For this project, the factory should stay small and practical, mainly to keep model construction clear and centralised.

Conclusion and Suggestions

5.1 Conclusion

This report proposed the design of a HealthTech cardiopulmonary sound separation system as a prototype and proof of concept. The system is intended to accept mixed WAV audio, run cardiopulmonary sound separation, and produce separate heart sound and lung sound output files. It supports the main user workflow of upload, validation, separation, preview, download, and processing history while keeping the implementation practical for a student software design project.

The proposed architecture is important because the machine learning model is only one part of the complete system. FastAPI provides the backend API layer for handling upload, separation, result, download, and history requests. NeoSSNet provides the core separation model for generating heart and lung audio outputs. SQLite stores structured metadata such as uploaded audio records, model details, separation jobs, result paths, evaluation metrics, and logs. Local storage keeps the actual uploaded WAV files, separated output files, and model weights, which avoids storing large audio content directly inside the database.

The selected design patterns strengthen the proposed design. The Facade Pattern simplifies the separation workflow by allowing the router to call a high-level service instead of directly managing preprocessing, inference, storage, logging, and database updates. The Strategy Pattern supports future algorithm flexibility by allowing the separation model to be represented through a common interface. The Factory Method Pattern centralises the creation of model or algorithm objects so that model selection and construction are not scattered across the codebase.

Overall, the proposed design demonstrates how machine learning, database persistence, local file storage, and web interaction can be organised into a reusable software system. The report does not claim clinical validation or real hospital deployment. Instead, it presents a clear and maintainable software design suitable for assignment submission, demonstration, and future implementation improvement.

5.2 Suggestions / Future Work

Several improvements can be considered after the core prototype is working:

1. **Improve frontend usability:** The user interface can be refined with clearer progress indicators, better error messages, improved history filtering, and more polished audio playback controls.
2. **Add more datasets and noise conditions:** Future testing can include broader cardiopulmonary recordings, different noise levels, and additional real-world recording conditions to better understand model behaviour.
3. **Add asynchronous or background processing:** Longer audio files or slower inference tasks can be handled using background jobs so that the web request does not need to wait synchronously for the full separation process.
4. **Add authentication if needed:** If the system is extended beyond a local demonstration, user login and role-based access can be added to protect uploaded files and processing history.
5. **Add algorithms through Strategy:** Alternative approaches, baseline methods, or future ONNX-based versions can be added as new strategies without changing the main router workflow.
6. **Improve evaluation metrics and reporting:** The system can provide clearer metric reports such as SNR, SDR, SI-SDR, MSE, or MAE where suitable reference data is available.
7. **Validate on broader real-world recordings:** Further validation should be performed on more varied recordings before making any claim about robustness or clinical usefulness.

In conclusion, the project provides a strong foundation for a practical software design around cardiopulmonary sound separation. Its main value is not only the use of NeoSSNet, but also the structured design that makes upload handling, inference, storage, logging, result viewing, and future extension easier to understand and maintain.

Bibliography / References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Poh, Y. Y., Grooby, E., Tan, K., Zhou, L., King, A., Ramanathan, A., Malhotra, A., Harandi, M., & Marzbanrad, F. (2024). NeoSSNet: Real-time neonatal chest sound separation using deep learning. *IEEE Open Journal of Engineering in Medicine and Biology*, *5*, 345–352. https://doi.org/10.1109/OJEMB.2024.3401571

Sun, W., Zhang, Y., & Chen, F. (2024). Research on heart and lung sound separation method based on DAE–NMF–VMD. *EURASIP Journal on Advances in Signal Processing*, *2024*, Article 59. https://doi.org/10.1186/s13634-024-01152-0

Zhao, Q., Geng, S., Wang, B., Sun, Y., Nie, W., Bai, B., Yu, C., Zhang, F., Tang, G., Zhang, D., Zhou, Y., Liu, J., & Hong, S. (2024). Deep learning in heart sound analysis: From techniques to clinical applications. *Health Data Science*, *4*, Article 0182. https://doi.org/10.34133/hds.0182